

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

Ho Chi Minh City University of Technology

Faculty of Computer Science and Engineering



COMPUTER NETWORKS - LAB (CO3094)

Laboratory

Report Lab 2

Instructor: Nguyen Thanh Nhan

Semester: 251

Class: CC06

Student: Le Pham Tien Long – 2352688

HO CHI MINH CITY, OCTORBER 2025



Contents

Table of Contents	1
1 Transport Layer	2
1.1 Wireshark Lab: TCP	2
1.1.1 A first look at the captured trace	2
1.1.2 TCP Basics	2
1.1.3 TCP congestion control in action	6
1.2 Wireshark Lab: UDP	7
2 Network/Internet Layer	9
2.1 Wireshark Lab: IP	9
2.1.1 Basic IPv4	9
2.1.2 Fragmentation	12
2.1.3 IPv6	14
2.2 Wireshark Lab: DHCP	15
2.3 Wireshark Lab: NAT	18
Code Availability	21

1 Transport Layer

1.1 Wireshark Lab: TCP

1.1.1 A first look at the captured trace

In this experiment, I analyzed TCP segments captured during a file transfer session to `gaia.cs.umass.edu`. The objective is to understand TCP connection establishment, data transfer mechanisms, and the fundamental fields within TCP segments. The trace file contains packets exchanged between a client and the server during the upload of `alice.txt` file using HTTP POST.

Question 1. What is the IP address and TCP port number used by the client computer (source) that is transferring the `alice.txt` file to `gaia.cs.umass.edu`?

As shown in Figure 1, the source IP address of the client computer is `192.168.1.10`, and the TCP source port number is `59906`. These values identify the client endpoint in this TCP connection. The source port is dynamically assigned by the operating system from the ephemeral port range, while the IP address represents the client's network interface on the local network.

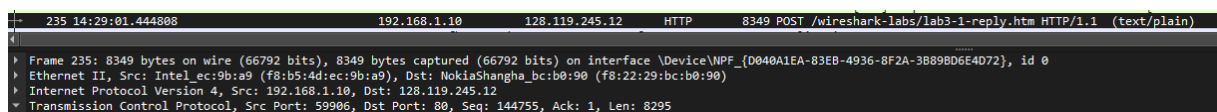


Figure 1: Wireshark capture showing the client IP address and TCP port number used for transferring `alice.txt` to `gaia.cs.umass.edu`. The source address `192.168.1.10` and source port `59906` are visible in the packet details.

Question 2. What is the IP address of `gaia.cs.umass.edu`? On what port number is it sending and receiving TCP segments for this connection?

From Figure 1, the destination IP address of `gaia.cs.umass.edu` is `128.119.245.12`, and it is using port `80` for both sending and receiving TCP segments. Port `80` is the standard well-known port for HTTP services, which aligns with the fact that this trace involves an HTTP POST request to upload the file to the web server.

1.1.2 TCP Basics

Question 3. What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and `gaia.cs.umass.edu`? What is it in this TCP segment that identifies the segment as a SYN segment? Will the TCP receiver in this session be able to use Selective Acknowledgments?

The initial TCP SYN segment has a sequence number of `0` (relative sequence number). As illustrated in Figure 2, this segment is identified as a SYN segment by the **SYN flag** being set to `1` in the TCP header flags field. Additionally, by examining the TCP options in the captured segment, we can observe that Selective Acknowledgment (SACK) is permitted, meaning the TCP receiver will be able to use Selective Acknowledgments during this session. This is indicated by the presence of the SACK-permitted option in the SYN segment.

Question 4. What is the sequence number of the SYNACK segment sent by `gaia.cs.umass.edu` to the client computer in reply to the SYN? What is it in the segment that identifies the segment as a SYNACK segment? What is the value of the Acknowledgement field in the SYNACK segment? How did `gaia.cs.umass.edu` determine that value?

As shown in Figure 3, the SYNACK segment sent by the server has a sequence number of `0` (relative). This segment is identified as a SYNACK by having both the **SYN flag** and the **ACK flag** set to `1`. The acknowledgement number field in the SYNACK segment contains the value `1`. The server determined this value by taking the initial sequence number received in the client's SYN segment (which was `0`) and adding `1` to it, following the standard TCP three-way handshake protocol where each byte and control flag consumes one sequence number.

```

69 14:29:00.425604 192.168.1.10 128.119.245.12 TCP 66 59906 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
Frame 69: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{0040A1EA-83EB-4936-8F2A-3889D06E4D72}, id 0
Ethernet II, Src: Intel_Ec:9b:a9 (f8:b5:4d:ec:9b:a9), Dst: NokiaShangha_bc:b0:90 (f8:22:29:bc:b0:90)
Internet Protocol Version 4, Src: 192.168.1.10, Dst: 128.119.245.12
Transmission Control Protocol, Src Port: 59906, Dst Port: 80, Seq: 0, Len: 0
Source Port: 59906
Destination Port: 80
[Stream index: 9]
[Stream Packet Number: 1]
[Conversation completeness: Incomplete, DATA (15)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 3090459576
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment Number (raw): 0
1000 .... = Header Length: 32 bytes (8)
Flags: 0x002 (SYN)
0000 .... = Reserved: Not set
...0 .... = Accurate ECN: Not set
....0... = Congestion Window Reduced: Not set
....0... = ECH-Echo: Not set
....0... = Urgent: Not set
....0... = Acknowledgment: Not set
....0... = Push: Not set
....0... = Reset: Not set
...1... = Syn: Set
....0... = Fin: Not set
[TCP Flags: .....S-]
Window: 65535
[Calculated window size: 65535]
Checksum: 0x375d [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted
  TCP Option - Maximum segment size: 1460 bytes
  TCP Option - No-Operation (NOP)
  TCP Option - Window scale: 8 (multiply by 256)
  TCP Option - No-Operation (NOP)
  TCP Option - No-Operation (NOP)
  TCP Option - SACK permitted
    Kind: SACK Permitted (4)
    Length: 2
  [Timestamps]

```

Figure 2: TCP SYN segment initiating the connection. The figure shows the sequence number field set to 0, the SYN flag set to 1, and the SACK-permitted option in the TCP options field.

```

119 14:29:00.680163 128.119.245.12 192.168.1.10 TCP 66 80 → 59906 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1412 SACK_PERM WS=128
Frame 119: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{0040A1EA-83EB-4936-8F2A-3889D06E4D72}, id 0
Ethernet II, Src: NokiaShangha_bc:b0:90 (f8:22:29:bc:b0:90), Dst: Intel_Ec:9b:a9 (f8:b5:4d:ec:9b:a9)
Internet Protocol Version 4, Src: 128.119.245.12, Dst: 192.168.1.10
Transmission Control Protocol, Src Port: 80, Dst Port: 59906, Seq: 0, Ack: 1, Len: 0
Source Port: 80
Destination Port: 59906
[Stream index: 9]
[Stream Packet Number: 2]
[Conversation completeness: Incomplete, DATA (15)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 405302710
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment Number (raw): 3090459577
1000 .... = Header Length: 32 bytes (8)
Flags: 0x012 (SYN, ACK)
0000 .... = Reserved: Not set
...0 .... = Accurate ECN: Not set
....0... = Congestion Window Reduced: Not set
....0... = ECH-Echo: Not set
....0... = Urgent: Not set
....1... = Acknowledgment: Set
....0... = Push: Not set
....0... = Reset: Not set
...1... = Syn: Set
....0... = Fin: Not set
[TCP Flags: .....A.-S-]
Window: 29200
[Calculated window size: 29200]
Checksum: 0x0f58 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), Maximum segment size, No-Operation (NOP), No-Operation (NOP), SACK permitted, No-Operation (NOP), Window scale
  TCP Option - Maximum segment size: 1412 bytes
  TCP Option - No-Operation (NOP)
  TCP Option - No-Operation (NOP)
  TCP Option - SACK permitted
    Kind: SACK Permitted (4)
    Length: 2
  TCP Option - No-Operation (NOP)
  TCP Option - Window scale: 7 (multiply by 128)
  [Timestamps]
  [SEQ/ACK analysis]

```

Figure 3: TCP SYNACK segment from the server. Both SYN and ACK flags are set to 1, the sequence number is 0, and the acknowledgement number is 1, confirming receipt of the client's SYN.

Question 5. What is the sequence number of the TCP segment containing the header of the HTTP POST command? How many bytes of data are contained in the payload (data) field of this TCP segment? Did all of the data in the transferred file `alice.txt` fit into this single segment?

The TCP segment containing the HTTP POST command header has a sequence number of 1, as illustrated in Figure 4. The payload (data field) of this segment contains 730 bytes. No, all the data in the transferred file `alice.txt` did not fit into this single segment. The file required multiple TCP segments for complete transmission, as evidenced by subsequent data-carrying segments in the trace file.

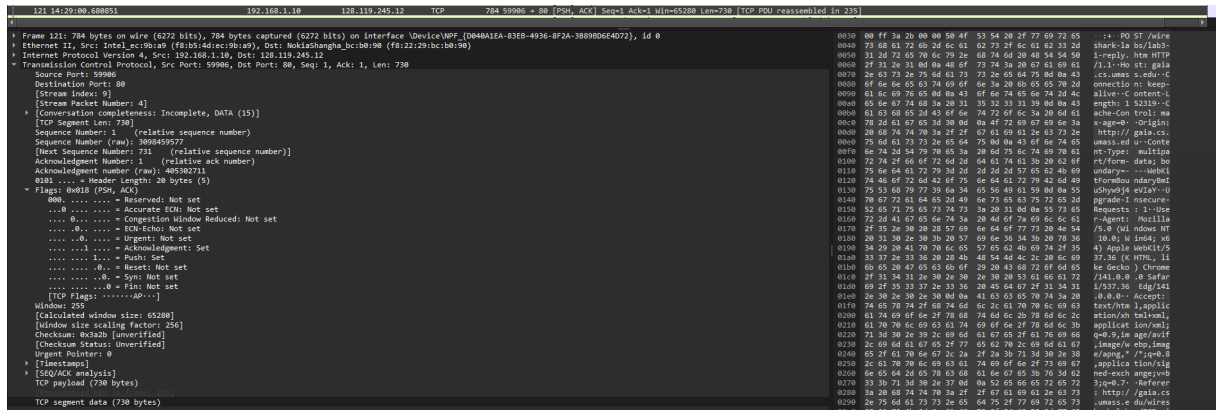


Figure 4: TCP segment containing the HTTP POST command. The sequence number is 1, and the segment length field shows 730 bytes of payload data including the HTTP headers and initial file content.

Question 6. Consider the TCP segment containing the HTTP "POST" as the first segment in the data transfer part of the TCP connection. At what time was the first segment sent? At what time was the ACK for this first data-containing segment received? What is the RTT for this first data-containing segment? What is the *EstimatedRTT* value after the ACK for the second data-carrying segment is received?

By examining the timestamp information in the captured trace:

- The first data-carrying segment (containing the HTTP POST) was sent at time 14:29:00.680851 seconds.
- The ACK for this first data-containing segment was received at time 14:29:00.934494 seconds.
- The Round-Trip Time (RTT) for this first data-containing segment is calculated as:

$$RTT = 14 : 29 : 00.934494 - 14 : 29 : 00.680851 = 0.253643 \text{ seconds}$$

- The *EstimatedRTT* value after the ACK for the second data-carrying segment is received can be computed using the standard TCP RTT estimation formula with $\alpha = 0.125$:

$$\text{EstimatedRTT} = 0.875 \times 0.255278 + 0.125 \times 0.253643 = 0.255073625 \text{ seconds}$$

Question 7. What is the length (header plus payload) of each of the first four data-carrying TCP segments?

By examining the captured segments, the lengths of the first four data-carrying TCP segments are as follows:

- First data-carrying TCP segment: 730 bytes
- Second data-carrying TCP segment: 12708 bytes
- Third data-carrying TCP segment: 24004 bytes
- Fourth data-carrying TCP segment: 2824 bytes

These values represent the total segment size including both the TCP header and the payload data.

Question 8. What is the minimum amount of available buffer space advertised to the client by `gaia.cs.umass.edu` among these first four data-carrying TCP segments? Does the lack of receiver buffer space ever throttle the sender for these first four data carrying segments?

The minimum amount of available buffer space advertised by the server to the client in the receive window field among the first four data-carrying TCP segments is 730 bytes. However, the lack of receiver buffer space does not throttle the sender during these first four data-carrying segments. The transmission proceeds smoothly without being limited by the advertised window size.

Question 9. Are there any retransmitted segments in the trace file? What did you check for (in the trace) in order to answer this question?

No, there are no retransmitted segments in the trace file. To verify this, I checked the sequence numbers of all TCP segments in the trace file. All sequence numbers are monotonically increasing without any repetition, which indicates that no segments needed to be retransmitted. If retransmissions had occurred, we would observe duplicate sequence numbers in the trace, which is not the case here.

Question 10. How much data does the receiver typically acknowledge in an ACK among the first ten data-carrying segments sent from the client to `gaia.cs.umass.edu`? Can you identify cases where the receiver is ACKing every other received segment (see Table 3.2 in the text) among these first ten data-carrying segments?

Table 1 presents the acknowledged sequence numbers and the amount of data acknowledged by each ACK for the first ten data-carrying segments.

ACK Number	Acknowledged Sequence	Data Acknowledged (bytes)
Segment 1	731	731
Segment 2	12027	11296
Segment 3	13439	1412
Segment 4	14851	1412
Segment 5	21911	7060
Segment 6	28971	7060
Segment 7	30383	1412
Segment 8	37443	7060
Segment 9	40267	2824
Segment 10	44503	4236

Table 1: Analysis of acknowledged sequence numbers for the first ten ACK segments. The "Data Acknowledged" column shows the difference between consecutive acknowledged sequence numbers, representing the amount of data received by the server between ACKs.

The difference between consecutive acknowledged sequence numbers indicates the amount of data received by the server between those two ACKs. By analyzing the pattern in Table 1, we can identify cases where the receiver is ACKing every other received segment rather than acknowledging each segment individually, which is a common TCP optimization technique called delayed acknowledgment.

Question 11. What is the throughput (bytes transferred per unit time) for the TCP connection? Explain how you calculated this value.

To calculate the throughput, I identified the first and last segments of the data transfer. As shown in Figure 5, the first TCP data segment (packet number 121) has a sequence number of 1 byte. Figure 6 shows the last ACK (packet number 265) with an acknowledged sequence number of 153050 bytes.

The total amount of data transmitted is calculated as:

$$\text{Total Data} = 153050 - 1 = 153049 \text{ bytes}$$

The transmission time is computed from the timestamps:

$$\text{Transmission Time} = 1.696593 - 0.680851 = 1.015742 \text{ seconds}$$

Therefore, the throughput for this TCP connection is:

$$\text{Throughput} = \frac{153049}{1.015742} \approx 150.678 \text{ KByte/sec}$$

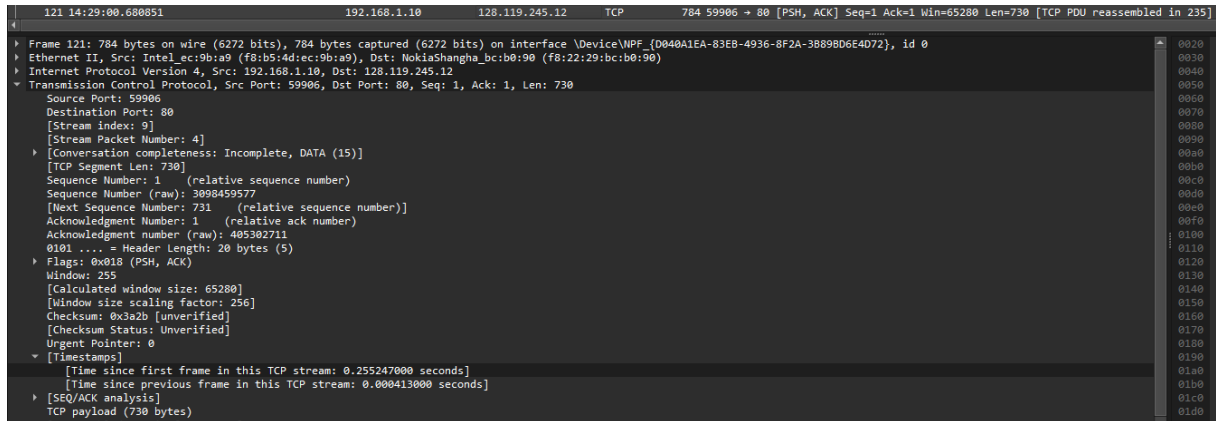


Figure 5: The first TCP data segment (packet 121) showing sequence number 1 and timestamp 0.680851 seconds, marking the beginning of the file transfer.

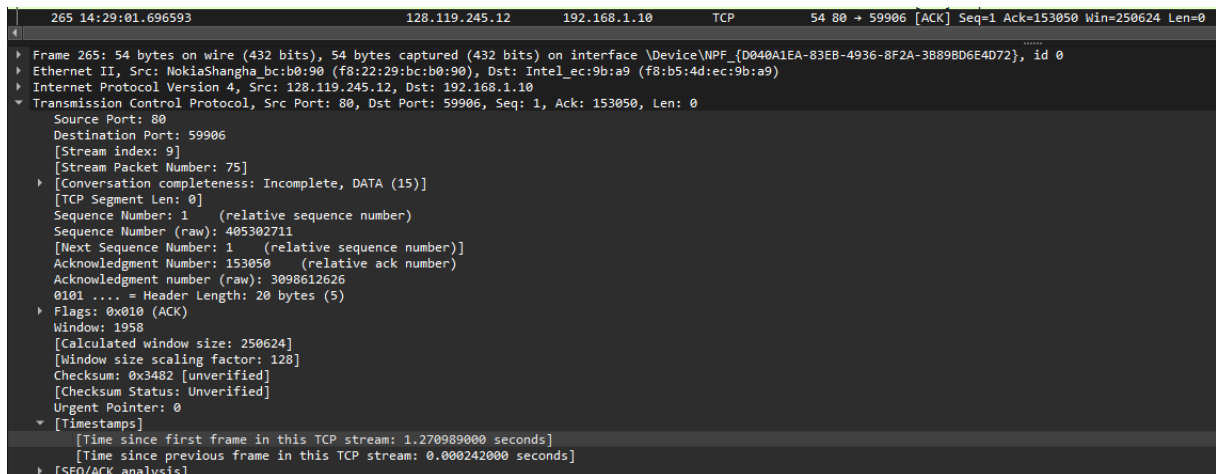


Figure 6: The final ACK segment (packet 265) with acknowledged sequence number 153050 and timestamp 1.696593 seconds, indicating successful receipt of all transmitted data.

1.1.3 TCP congestion control in action

Question 12. Use the Time-Sequence-Graph(Stevens) plotting tool to view the sequence number versus time plot of segments being sent from the client to the gaia.cs.umass.edu server. Consider the "fleets" of packets sent around $t = 0.025$, $t = 0.053$, $t = 0.082$ and $t = 0.1$. Comment on whether this looks as if TCP is in its slow start phase, congestion avoidance phase or some other phase.

The Time-Sequence-Graph plot reveals that the number of packets transmitted by the client increases exponentially over time. This exponential growth pattern is characteristic of TCP's slow start phase, where the congestion window doubles with each round-trip time until a threshold is reached. The "fleets" of packets observed at regular intervals represent the transmission of multiple segments within a single congestion window before waiting for acknowledgments.

Question 13. These "fleets" of segments appear to have some periodicity. What can you say about the period?

The observed periodicity of the segment "fleets" corresponds to the Round-Trip Time (RTT) of the connection. Each fleet represents one window's worth of data being transmitted, and the spacing between fleets equals the time required for acknowledgments to return to the sender, allowing the next window of data to be sent.

Question 14. Answer each of two questions above for the trace that you have gathered when you transferred a file from your computer to gaia.cs.umass.edu.

Figure 7 shows the Time-Sequence-Graph from my own packet capture. The plot demonstrates similar behavior to the provided trace, with the number of packets transmitted increasing exponentially over

time, confirming that TCP is operating in the slow start phase. The "fleets" of segments also exhibit regular periodicity corresponding to the Round-Trip Time of my connection to the server.

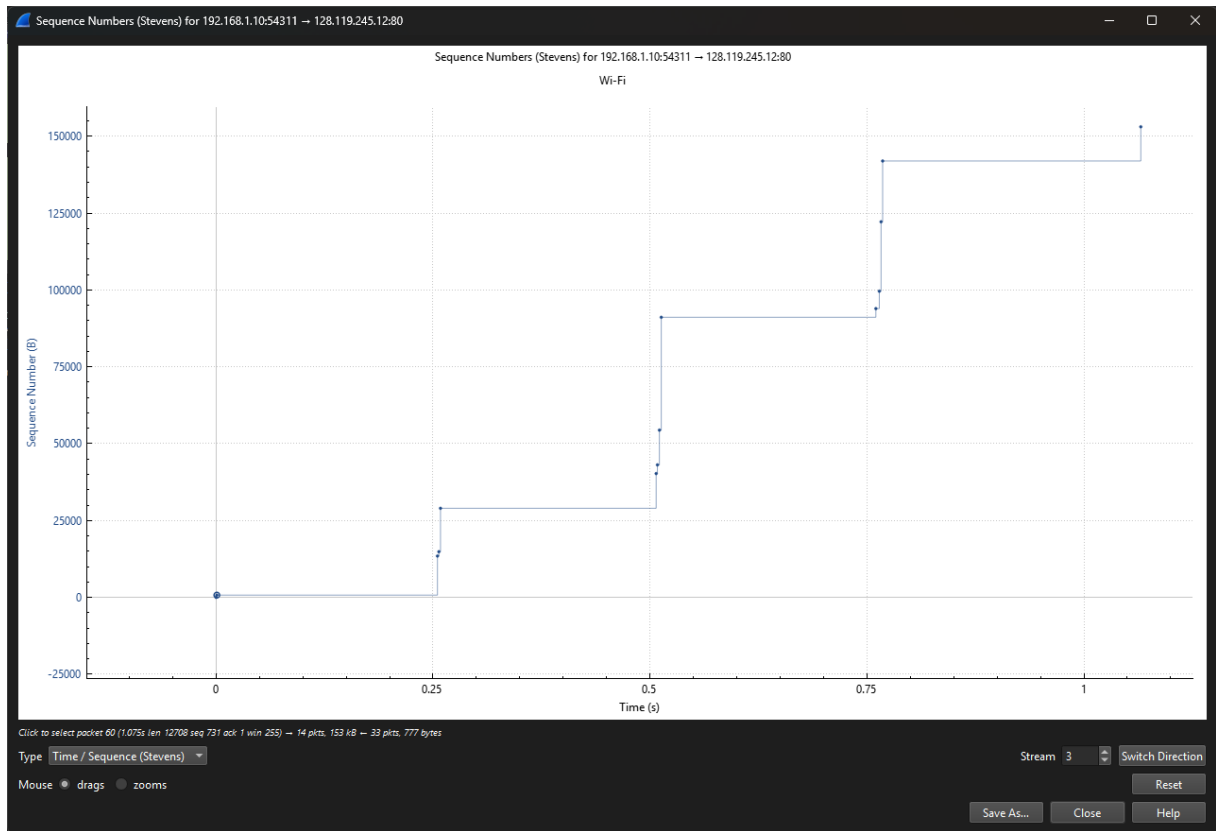


Figure 7: Time-Sequence-Graph (Stevens style) for my file transfer to *gaia.cs.umass.edu*. The plot shows exponential growth in sequence numbers, indicating TCP slow start phase, with periodic "fleets" of packets spaced by RTT intervals.

1.2 Wireshark Lab: UDP

In this experiment, I analyzed UDP datagrams to understand the structure and characteristics of the User Datagram Protocol. Unlike TCP, UDP is a connectionless, unreliable transport protocol that provides minimal overhead for applications that do not require guaranteed delivery. The trace file was captured during DNS queries and other UDP-based communications.

Question 1. Select the first UDP segment in your trace. What is the packet number of this segment in the trace file? What type of application-layer payload or protocol message is being carried in this UDP segment? Look at the details of this packet in Wireshark. How many fields are there in the UDP header? What are the names of these fields?

The first UDP segment in the trace file is packet number 1. As shown in Figure 8, this segment carries a DNS query as its application-layer payload. The UDP header contains exactly 4 fields: **Source Port**, **Destination Port**, **Length**, and **Checksum**. This simple header structure reflects UDP's lightweight design philosophy.

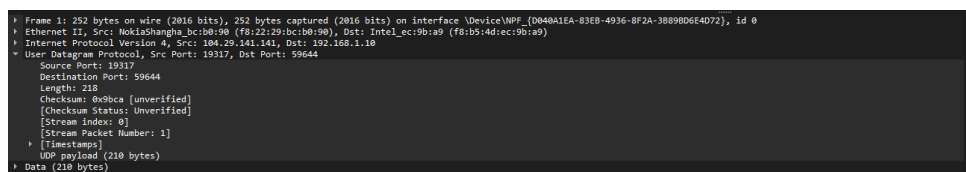


Figure 8: The first UDP segment in the trace (packet number 1). The expanded UDP header shows all four fields: Source Port, Destination Port, Length, and Checksum. The payload contains a DNS query message.

Question 2. By consulting the displayed information in Wireshark's packet content field for this packet (or by consulting the textbook), what is the length (in bytes) of each of the UDP header fields?

Each field in the UDP header is **2 bytes** (16 bits) long. This fixed size contributes to UDP's efficiency, as the total header is only 8 bytes, significantly smaller than TCP's minimum 20-byte header.

Question 3. The value in Length field is the length of what?

The value in the Length field represents the total length of the UDP datagram, including both the 8-byte UDP header and the payload data. As demonstrated in Figure 8, the Length field shows **218 bytes**, which equals **8 bytes** (header) + **210 bytes** (payload data). This field allows the receiver to determine where the UDP datagram ends.

Question 4. What is the maximum number of bytes that can be included in a UDP payload?

The maximum UDP payload size is determined by the maximum value the 16-bit Length field can hold, minus the 8-byte UDP header. The calculation is:

$$\text{Max UDP Payload} = 2^{16} - 1 - 8 = 65536 - 1 - 8 = 65527 \text{ bytes}$$

However, in practice, the maximum payload is often limited by the Maximum Transmission Unit (MTU) of the underlying network to avoid IP fragmentation.

Question 5. What is the largest possible source port number?

Since the source port field is 16 bits long, the largest possible source port number is:

$$2^{16} - 1 = 65535$$

This range includes well-known ports (0-1023), registered ports (1024-49151), and dynamic/private ports (49152-65535).

Question 6. What is the protocol number for UDP? Give your answer in decimal notation.

As shown in Figure 9, the protocol number for UDP in the IP header is **17** (decimal). This value in the Protocol field of the IP header allows the receiving system to identify that the payload should be passed to the UDP handler.

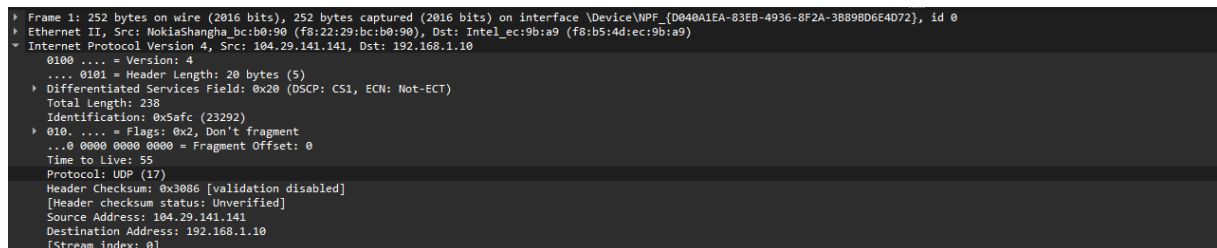


Figure 9: IP header showing the Protocol field set to 17, identifying the payload as a UDP datagram. This protocol number is standardized by IANA.

Question 7. Examine the pair of UDP packets in which your host sends the first UDP packet and the second UDP packet is a reply to this first UDP packet. (Hint: for a second packet to be sent in response to a first packet, the sender of the first packet should be the destination of the second packet). What is the packet number of the first of these two UDP segments in the trace file? What is the value in the source port field in this UDP segment? What is the value in the destination port field in this UDP segment? What is the packet number of the second of these two UDP segments in the trace file? What is the value in the source port field in this second UDP segment? What is the value in the destination port field in this second UDP segment? Describe the relationship between the port numbers in the two packets.

Figure 10 illustrates a request-response pair of UDP datagrams. The first UDP segment is packet number 135, where the source IP is 192.168.1.10 and the destination IP is 8.8.8.8 (Google's DNS server). The second UDP segment is packet number 136, which reverses these addresses with source IP 8.8.8.8 and destination IP 192.168.1.10.

The relationship between port numbers follows the client-server model: the source port in the request becomes the destination port in the response, and vice versa. This port swapping allows the response to be correctly delivered back to the originating application.

135	11:58:06.672771	192.168.1.10	8.8.8.8	DNS	71 Standard query 0x0003 AAAA www.nyu.com
136	11:58:07.022547	8.8.8.8	192.168.1.10	DNS	130 Standard query response 0x0003 AAAA www.nyu.com SOA ns1.eftydns.com

Figure 10: A pair of UDP datagrams showing request (packet 135) and response (packet 136). The source and destination IP addresses and port numbers are swapped between the two packets, demonstrating UDP's connectionless request-response pattern.

2 Network/Internet Layer

2.1 Wireshark Lab: IP

2.1.1 Basic IPv4

In this experiment, I examined IP datagrams captured during a `traceroute` command execution to `gaia.cs.umass.edu`. The objective is to understand the structure of IPv4 headers, the behavior of key fields such as TTL and Identification, and the mechanism of IP fragmentation. The trace file contains both outgoing UDP segments encapsulated in IP datagrams and incoming ICMP responses from intermediate routers.

Question 1. Select the first UDP segment sent by your computer via the `traceroute` command to `gaia.cs.umass.edu`. (Hint: this is 44th packet in the trace file in the `ip-wireshark-trace1-1.pcapng` file in footnote 2). Expand the Internet Protocol part of the packet in the packet details window. What is the IP address of your computer?

As illustrated in Figure 11, the first UDP segment sent by my computer via the `traceroute` command shows the source IP address as `192.168.86.61`. This is the IPv4 address assigned to my computer's network interface on the local network.

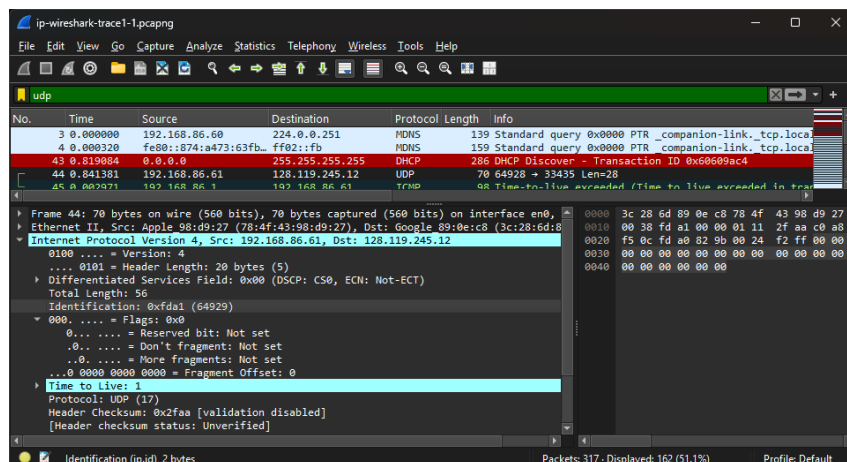


Figure 11: The first UDP segment sent via `traceroute` to `gaia.cs.umass.edu`. The Internet Protocol section shows the source IP address `192.168.86.61` and various IPv4 header fields.

Question 2. What is the value in the time-to-live (TTL) field in this IPv4 datagram's header?

From Figure 11, the value in the time-to-live (TTL) field is 1. The `traceroute` utility works by systematically increasing the TTL value for successive probes, starting with TTL=1. This causes the first router to discard the packet and send back an ICMP Time Exceeded message, revealing the router's identity.

Question 3. What is the value in the upper layer protocol field in this IPv4 datagram's header?

From Figure 11, the upper layer protocol field contains the value UDP (17). This indicates that the payload of this IP datagram is a UDP segment, which is consistent with how `traceroute` typically operates on many systems.

Question 4. How many bytes are in the IP header?

From Figure 11, the IP header length is 20 bytes. This is the minimum size for an IPv4 header without options, consisting of the standard fixed fields.

Question 5. How many bytes are in the payload of the IP datagram? Explain how you determined the number of payload bytes.

There are 36 bytes in the payload of the IP datagram. This value is determined by subtracting the

header length from the total length field. From Figure 11, the total length is 56 bytes and the header length is 20 bytes, therefore:

$$\text{Payload Size} = 56 - 20 = 36 \text{ bytes}$$

Question 6. Has this IP datagram been fragmented? Explain how you determined whether or not the datagram has been fragmented.

From Figure 11, this IP datagram has not been fragmented. This determination is based on two indicators: (1) the "More fragments" flag is **not set**, and (2) the "Fragment Offset" field is 0. If fragmentation had occurred, either the More fragments flag would be set (for all but the last fragment) or the Fragment Offset would be non-zero (for all but the first fragment).

Question 7. Which fields in the IP datagram always change from one datagram to the next within this series of UDP segments sent by your computer destined to 128.119.245.12, via traceroute? Why?

The fields that always change from one datagram to the next are the **Identification** field and the **Header Checksum** field. The Identification field changes because each datagram must have a unique identifier for potential reassembly if fragmentation occurs. The Header Checksum changes because it is recalculated based on the header contents, and since the Identification and TTL fields change, the checksum must be updated accordingly.

Question 8. Which fields in this sequence of IP datagrams (containing UDP segments) stay constant? Why?

The fields that remain constant throughout the sequence are: **Version** (IPv4), **Header Length**, **Source IP address** (sending from the same computer), **Destination IP address** (all packets destined to the same server), and **Upper Layer Protocol** (all carrying UDP). These fields stay constant because they represent unchanging characteristics of the communication: the IP version being used, the source and destination endpoints, and the transport protocol.

```
No.    Time           Source           Destination      Protocol Length Info
 44 0.841381      192.168.86.61    128.119.245.12   UDP           70      64928 → 33435 Len=28
Frame 44: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface en0, id 0
Ethernet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Google_89:0e:c8 (3c:28:6d:89:0e:c8)
Internet Protocol Version 4, Src: 192.168.86.61, Dst: 128.119.245.12
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 56
Identification: 0xfda1 (64929)
000. .... = Flags: 0x0
 0... .... = Reserved bit: Not set
 .0... .... = Don't fragment: Not set
..0... .... = More fragments: Not set
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 1
[Expert Info (Note/Sequence): "Time To Live" only 1]
Protocol: UDP (17)
Header Checksum: 0x2faa [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.168.86.61
Destination Address: 128.119.245.12
[Stream index: 4]
User Datagram Protocol, Src Port: 64928, Dst Port: 33435
Data (28 bytes)
0000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
No.    Time           Source           Destination      Protocol Length Info
 48 0.000422      192.168.86.61    128.119.245.12   UDP           70      64928 → 33436 Len=28
Frame 48: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface en0, id 0
Ethernet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Google_89:0e:c8 (3c:28:6d:89:0e:c8)
Internet Protocol Version 4, Src: 192.168.86.61, Dst: 128.119.245.12
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 56
Identification: 0xfda2 (64930)
000. .... = Flags: 0x0
 0... .... = Reserved bit: Not set
 .0... .... = Don't fragment: Not set
..0... .... = More fragments: Not set
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 1
[Expert Info (Note/Sequence): "Time To Live" only 1]
Protocol: UDP (17)
Header Checksum: 0x2fa9 [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.168.86.61
Destination Address: 128.119.245.12
[Stream index: 4]
User Datagram Protocol, Src Port: 64928, Dst Port: 33436
Data (28 bytes)
0000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
No.    Time           Source           Destination      Protocol Length Info
 50 0.000865      192.168.86.61    128.119.245.12   UDP           70      64928 → 33437 Len=28
Frame 50: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface en0, id 0
Ethernet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Google_89:0e:c8 (3c:28:6d:89:0e:c8)
Internet Protocol Version 4, Src: 192.168.86.61, Dst: 128.119.245.12
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 56
Identification: 0xfda3 (64931)
```

Figure 12: A sequence of IP datagrams showing the Identification field incrementing by 1 for each successive packet. This sequential pattern helps ensure unique identification for fragmentation purposes.

Question 9. Describe the pattern you see in the values in the Identification field of the IP datagrams being sent by your computer.

As shown in Figure 12, the Identification field values increase sequentially by 1 for each successive datagram. This is a common pattern used by operating systems to assign unique identification numbers to outgoing IP datagrams.

Question 10. What is the upper layer protocol specified in the IP datagrams returned from the routers?

As illustrated in Figure 13, the upper layer protocol in the IP datagrams returned from the routers is ICMP (1). Routers send ICMP Time Exceeded messages in response to **traceroute** probes with expired TTL values, which is how **traceroute** discovers the path to the destination.

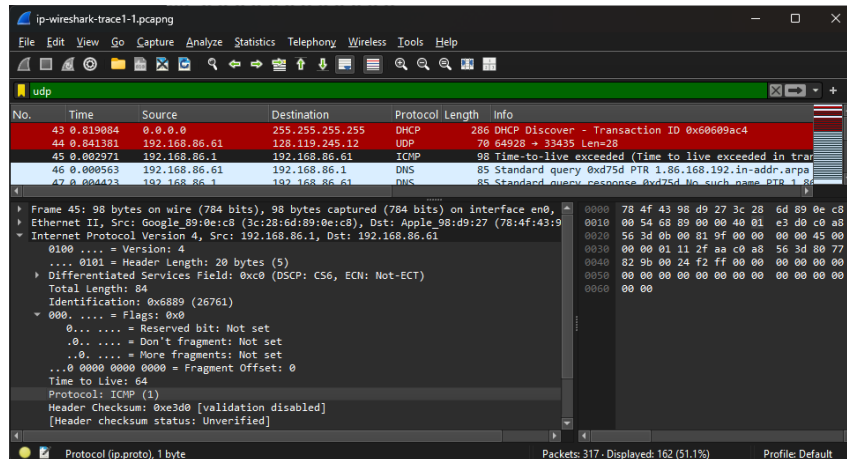


Figure 13: An ICMP response from a router showing the Protocol field set to 1 (ICMP). This Time Exceeded message is generated when a router discards a packet with TTL=0.



Figure 14: Multiple ICMP packets from various routers showing irregular Identification field values. Unlike the sequential pattern in outgoing packets, these values vary based on each router's implementation.

Question 11. Are the values in the Identification fields (across the sequence of all of ICMP packets from all of the routers) similar in behavior to your answer to question 9 above?

No, the Identification field values in the ICMP packets from the routers do not exhibit the same sequential pattern observed in the outgoing datagrams. As shown in Figure 14, the Identification values vary without a consistent pattern. This is because these ICMP responses are generated by different routers, each using its own identification numbering scheme.

Question 12. Are the values of the TTL fields similar, across all of ICMP packets from all of the routers? From Figure 14, the TTL values vary across ICMP packets from different routers. Each router sets its own initial TTL value when generating ICMP responses, which depends on the router's operating system and configuration. The TTL also decreases as the ICMP packet traverses back through intermediate routers to reach the source.

2.1.2 Fragmentation

Question 13. Find the first IP datagram containing the first part of the segment sent to 128.119.245.12 sent by your computer via the `traceroute` command to `gaia.cs.umass.edu`, after you specified that the `traceroute` packet length should be 3000. Has that segment been fragmented across more than one IP datagram?

Yes, when the `traceroute` packet length is specified as 3000 bytes, the segment has been fragmented across more than one IP datagram. This fragmentation occurs because the packet size exceeds the Maximum Transmission Unit (MTU) of the network path, which is typically 1500 bytes for Ethernet networks.

Question 14. What information in the IP header indicates that this datagram been fragmented?

As shown in Figure 15, the IP header field "Flags: 0x1, More fragments" indicates that this datagram has been fragmented. The "More fragments" flag being set to 1 signals that additional fragments follow this one to complete the original packet.

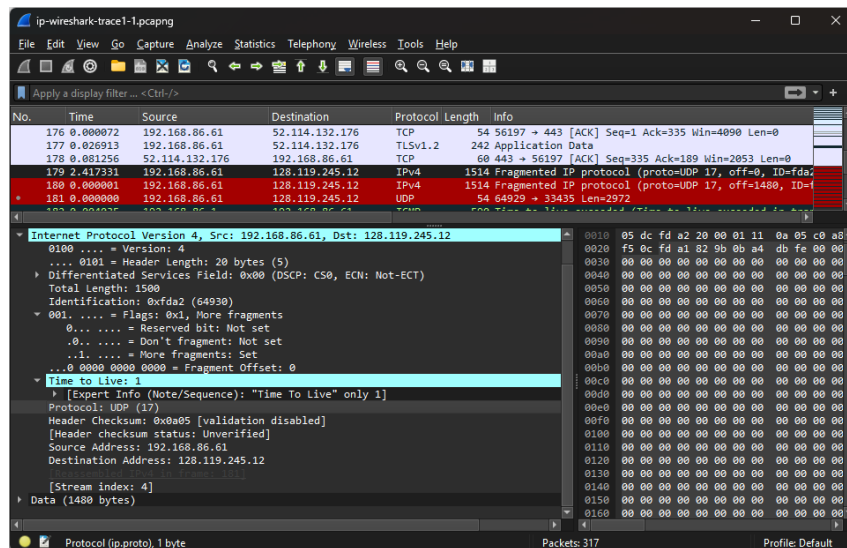


Figure 15: The first fragment of a fragmented UDP segment. The "More fragments" flag is set, and the Fragment Offset is 0, indicating this is the first piece of a larger datagram.

Question 15. What information in the IP header for this packet indicates whether this is the first fragment versus a latter fragment?

From Figure 15, the "Fragment Offset: 0" field indicates that this is the first fragment. The Fragment Offset specifies where in the original datagram this fragment belongs, measured in 8-byte units. A value of 0 means this fragment contains the beginning of the original packet.

Question 16. How many bytes are there in this IP datagram (header plus payload)?

From Figure 15, the total length of this IP datagram is 1500 bytes, which includes both the IP header and the payload. This size matches the typical Ethernet MTU, which is why fragmentation was necessary for the larger original packet.

Question 17. Now inspect the datagram containing the second fragment of the fragmented UDP

segment. What information in the IP header indicates that this is not the first datagram fragment? As shown in Figure 16, the "Fragment Offset: 1480" field indicates that this is not the first fragment. This non-zero offset value specifies that the data in this fragment begins at byte position $1480 \times 8 = 11840$ in the original unfragmented datagram.

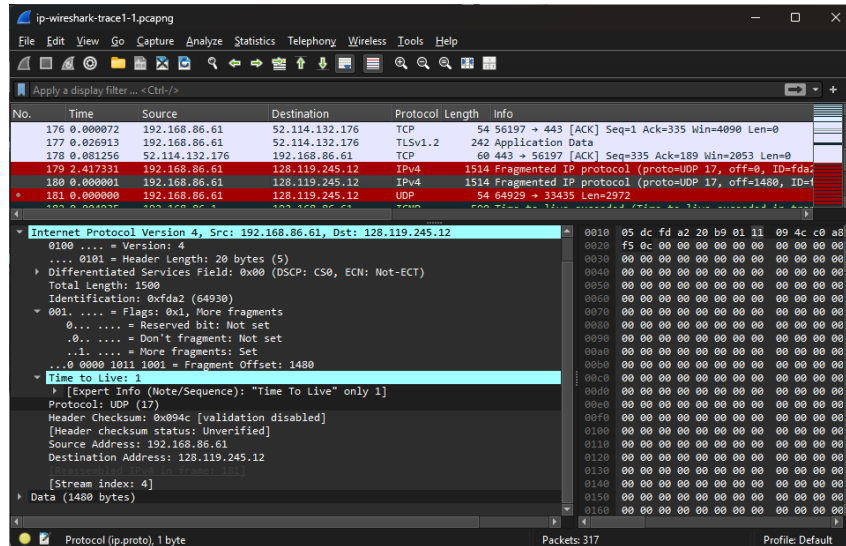


Figure 16: The second fragment of the fragmented UDP segment. The Fragment Offset value of 1480 indicates this fragment's position in the reassembly process, distinguishing it from the first fragment.

Question 18. What fields change in the IP header between the first and second fragment? Comparing Figure 15 and Figure 16, the fields that change between the first and second fragment are the Fragment Offset field and the Header Checksum field. The Fragment Offset changes to indicate the position of each fragment in the original datagram, and the Header Checksum is recalculated because the header contents have changed.

Question 19. Now find the IP datagram containing the third fragment of the original UDP segment. What information in the IP header indicates that this is the last fragment of that segment? As illustrated in Figure 17, the field "More fragments: Not set" indicates that this is the last fragment. When the More fragments flag is 0, it signals to the receiver that no additional fragments follow and reassembly can be completed.

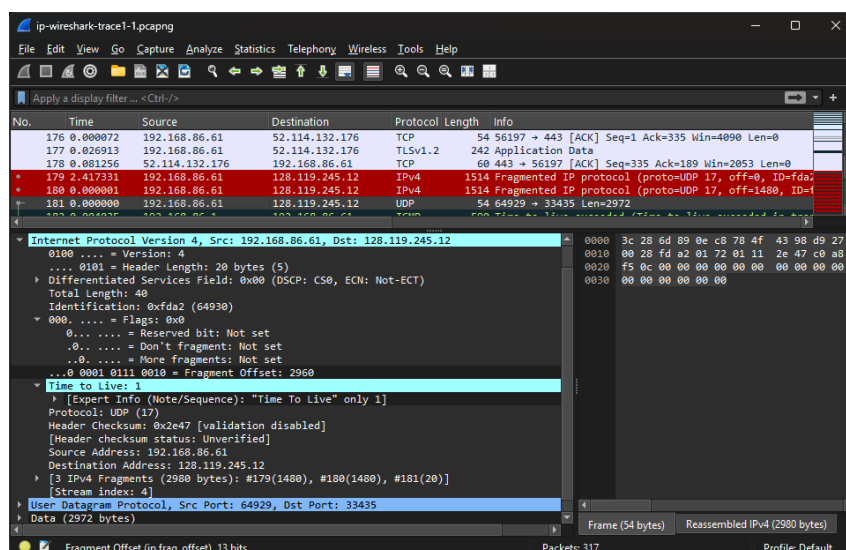


Figure 17: The final fragment of the fragmented UDP segment. The "More fragments" flag is not set, indicating this is the last piece needed to reassemble the original datagram.

2.1.3 IPv6

Question 20. What is the IPv6 address of the computer making the DNS AAAA request? This is the source address of the 20th packet in the trace. Give the IPv6 source address for this datagram in the exact same form as displayed in the Wireshark window.

As shown in Figure 18, the IPv6 source address is: 2601:193:8302:4620:215c:f5ae:8b40:a27a

This represents the globally routable IPv6 address assigned to the computer initiating the DNS AAAA query, which requests the IPv6 address of a domain name.

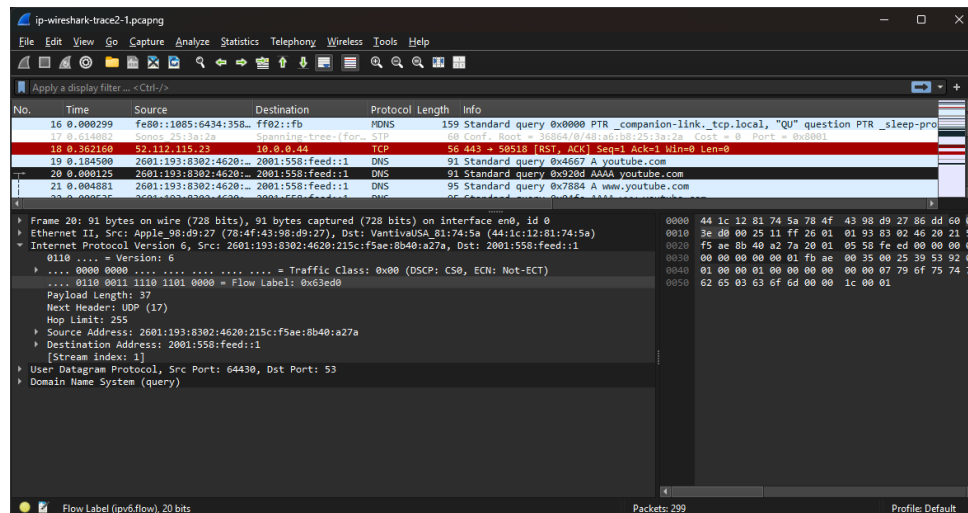


Figure 18: DNS AAAA request showing the IPv6 source address. The IPv6 header displays the full 128-bit address in colon-hexadecimal notation.

Question 21. What is the IPv6 destination address for this datagram? Give this IPv6 address in the exact same form as displayed in the Wireshark window.

From Figure 18, the IPv6 destination address is: 2001:558:feed::1

This address uses IPv6's zero compression notation (::) to represent consecutive groups of zeros, making the address more readable.

Question 22. What is the value of the flow label for this datagram?

From Figure 18, the flow label value is 0x63ed0. The flow label is a 20-bit field in the IPv6 header used to label sequences of packets for which the sender requests special handling, such as non-default quality of service or real-time traffic.

Question 23. How much payload data is carried in this datagram?

From Figure 18, the payload length is 37 bytes. In IPv6, the Payload Length field specifies the size of the payload following the IPv6 header, not including the header itself (unlike IPv4's Total Length field).

Question 24. What is the upper layer protocol to which this datagram's payload will be delivered at the destination?

From Figure 18, the Next Header field value is UDP (17). In IPv6, the Next Header field serves a similar purpose to IPv4's Protocol field, identifying the type of header that immediately follows the IPv6 header.

Question 25. How many IPv6 addresses are returned in the response to this AAAA request?

As shown in Figure 19, one IPv6 address is returned in response to the DNS AAAA query. The AAAA record type specifically requests the IPv6 address(es) associated with a domain name.

Question 26. What is the first of the IPv6 addresses returned by the DNS for youtube.com (in the ip-wireshark-trace2-1.pcapng trace file, this is also the address that is numerically the smallest)? Give this IPv6 address in the exact same shorthand form as displayed in the Wireshark window.

From Figure 19, the IPv6 address returned is: 2607:f8b0:4006:815::200e

This address uses IPv6's shorthand notation with zero compression (::) to represent the consecutive zero fields, which is the standard way to write IPv6 addresses concisely.

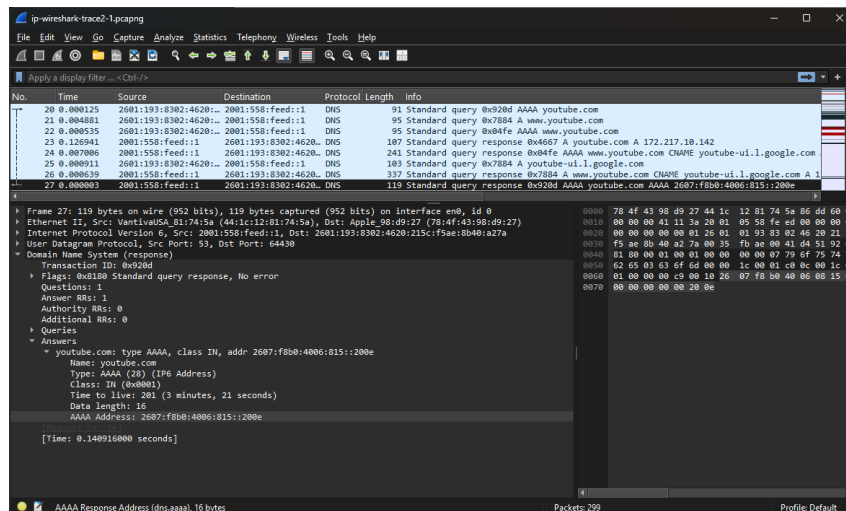


Figure 19: DNS AAAA response containing one IPv6 address. The answer section shows the AAAA record with the requested IPv6 address for the queried domain.

2.2 Wireshark Lab: DHCP

In this experiment, I analyzed the DHCP (Dynamic Host Configuration Protocol) message exchange captured during the process of obtaining an IP address from a DHCP server. DHCP operates at the application layer and uses UDP as its transport protocol. The four-message exchange (DISCOVER, OFFER, REQUEST, ACK) allows clients to dynamically obtain network configuration parameters including IP address, subnet mask, default gateway, and DNS servers.

Question 1. Is this DHCP Discover message sent out using UDP or TCP as the underlying transport protocol?

As shown in Figure 20, the DHCP Discover message is sent using UDP as the underlying transport protocol. This is evident from the protocol stack showing UDP wrapping the DHCP (BOOTP) message. DHCP uses UDP because it needs to function before a client has an IP address, and TCP would require connection establishment which necessitates having an IP address.

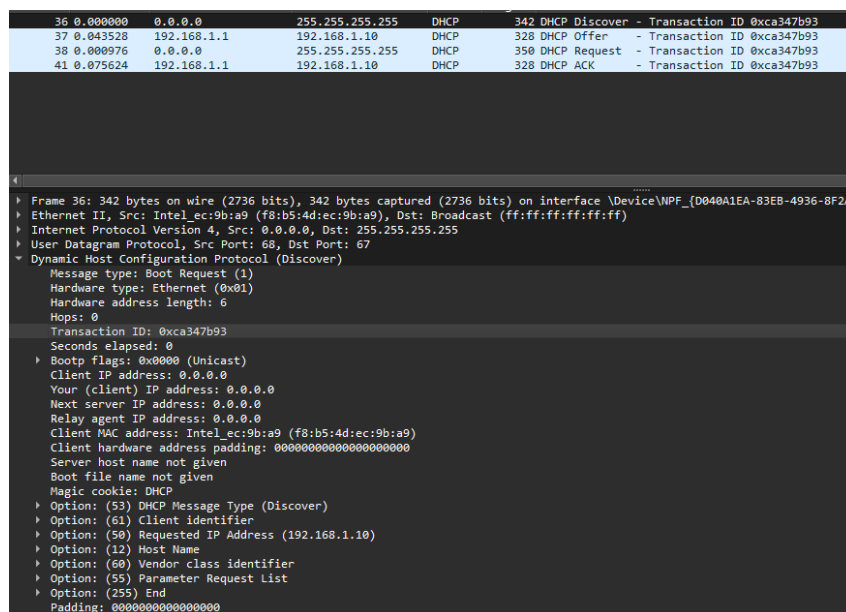


Figure 20: DHCP Discover message showing UDP as the transport protocol. The packet details display the protocol hierarchy: Ethernet → IP → UDP → DHCP, with source port 68 and destination port 67.

Question 2. What is the source IP address used in the IP datagram containing the Discover message?

Is there anything special about this address? Explain.

From Figure 20, the source IP address is 0.0.0.0. This is a special address that indicates the client does not yet have an assigned IP address. Since the DHCP Discover is the first message in the DHCP process, the client has no valid IP address to use as the source, so it uses the all-zeros address 0.0.0.0 as a placeholder.

Question 3. What is the destination IP address used in the datagram containing the Discover message. Is there anything special about this address? Explain.

From Figure 20, the destination IP address is 255.255.255.255. This is the limited broadcast address, which transmits the packet to all hosts on the local network segment. This broadcasting is necessary because the client does not know the IP address of the DHCP server, so it must broadcast the Discover message to reach any available DHCP servers on the subnet.

Question 4. What is the value in the transaction ID field of this DHCP Discover message?

From Figure 20, the transaction ID is 0xca347b93. This 32-bit identifier is randomly generated by the client and remains the same throughout the entire DHCP exchange (Discover, Offer, Request, ACK) to match responses with requests.

Question 5. Now inspect the options field in the DHCP Discover message. What are five pieces of information (beyond an IP address) that the client is suggesting or requesting to receive from the DHCP server as part of this DHCP transaction?

From Figure 20, five pieces of information requested by the client in the Parameter Request List are:

- Parameter Request List (specifying which network parameters the client wants)
- Maximum DHCP Message Size
- Client Identifier
- IP Address Lease Time
- Host Name

These options allow the client to communicate its capabilities and preferences to the DHCP server.

Question 6. How do you know that this Offer message is being sent in response to the DHCP Discover message you studied in questions 1-5 above?

The DHCP Offer message is matched to the corresponding Discover message by the **Transaction ID** field. Both messages share the same transaction ID value (0xca347b93), which confirms that this Offer is the server's response to the specific Discover message we examined earlier.

37 0.043528	192.168.1.1	192.168.1.10	DHCP	328 DHCP Offer	- Transaction ID 0xca347b93
38 0.000976	0.0.0.0	255.255.255.255	DHCP	350 DHCP Request	- Transaction ID 0xca347b93
41 0.075624	192.168.1.1	192.168.1.10	DHCP	328 DHCP ACK	- Transaction ID 0xca347b93


```

37: 328 bytes on wire (2624 bits), 328 bytes captured (2624 bits) on interface \Device\NPF_{D040A1EA-83EB-4936-8F2A}
38: Ethernet II, Src: NokiaShangha_bc:b0:90 (f8:22:29:bc:b0:90), Dst: Intel_ec:9b:a9 (f8:b5:4d:ec:9b:a9)
39: Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.10
40: User Datagram Protocol, Src Port: 67, Dst Port: 68
41: Dynamic Host Configuration Protocol (Offer)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0xca347b93
  Seconds elapsed: 0
  Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 192.168.1.10
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: Intel_ec:9b:a9 (f8:b5:4d:ec:9b:a9)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  Option: (53) DHCP Message Type (Offer)
  Option: (54) DHCP Server Identifier (192.168.1.1)
  Option: (51) IP Address Lease Time
  Option: (1) Subnet Mask (255.255.255.0)
  Option: (3) Router
  Option: (6) Domain Name Server
  Option: (58) Renewal Time Value
  Option: (59) Rebinding Time Value
  Option: (255) End
  
```

Figure 21: DHCP Offer message from the server. The source address 192.168.1.1 identifies the DHCP server, and the message contains the offered IP address and configuration parameters for the client.

Question 7. What is the source IP address used in the IP datagram containing the Offer message? Is there anything special about this address? Explain.

As shown in Figure 21, the source IP address is 192.168.1.1. This is the IP address of the DHCP server itself. Unlike the client which must use 0.0.0.0, the server already has a configured IP address and uses it as the source for its responses.

Question 8. What is the destination IP address used in the datagram containing the Offer message? Is there anything special about this address? Explain.

From Figure 21, the destination IP address is 192.168.1.10. This is the IP address being offered to the client. The DHCP server can send the Offer directly to this address (rather than broadcasting) because it knows the client's MAC address from the Discover message and can use it for link-layer delivery.

Question 9. Now inspect the options field in the DHCP Offer message. What are five pieces of information that the DHCP server is providing to the DHCP client in the DHCP Offer message?

From Figure 21, five pieces of information provided by the server are:

- Subnet Mask (defining the network portion of the IP address)
- Broadcast Address (for sending broadcasts on this network)
- Router (default gateway address)
- Domain Name (the DNS domain name for the network)
- Domain Name Server (DNS server address(es))

Question 10. What is the UDP source port number in the IP datagram containing the first DHCP Request message in your trace? What is the UDP destination port number being used?

As shown in Figure 22, the UDP source port is 68 and the destination port is 67. Port 68 is the standard DHCP client port, and port 67 is the standard DHCP server port. These well-known port numbers are defined in the DHCP specification.

38	0.000976	0.0.0.0	255.255.255.255	DHCP	350 DHCP Request	- Transaction ID 0xca347b93
41	0.075624	192.168.1.1	192.168.1.10	DHCP	328 DHCP ACK	- Transaction ID 0xca347b93


```

Frame 38: 350 bytes on wire (2800 bits), 350 bytes captured (2800 bits) on interface \Device\NPF_{D040A1EA-83EB-4936-8F2A}
Ethernet II, Src: Intel_ec:9b:a9 (f8:b5:4d:ec:9b:a9), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
User Datagram Protocol, Src Port: 68, Dst Port: 67
Dynamic Host Configuration Protocol (Request)

```

Figure 22: DHCP Request message showing UDP ports 68 (client) and 67 (server). This message confirms the client's acceptance of the offered IP address configuration.

Question 11. What is the source IP address in the IP datagram containing this Request message? Is there anything special about this address? Explain.

From Figure 22, the source IP address is 0.0.0.0. Even though the client has received an IP address offer, it has not yet been officially assigned or acknowledged by the server. Therefore, the client continues to use 0.0.0.0 as the source address until the DHCP ACK confirms the assignment.

Question 12. What is the destination IP address used in the datagram containing this Request message. Is there anything special about this address? Explain.

From Figure 22, the destination IP address is 255.255.255.255, the broadcast address. The Request is broadcast (rather than unicast to the server) because there may be multiple DHCP servers on the network, and broadcasting the Request allows all servers to know which offer was accepted and which server was selected.

Question 13. What is the value in the transaction ID field of this DHCP Request message? Does it match the transaction IDs of the earlier Discover and Offer messages?

The transaction ID is 0xca347b93, which matches the transaction IDs in both the Discover and Offer messages. This consistency allows all four messages (Discover, Offer, Request, ACK) to be associated

with the same DHCP transaction.

Question 14. What differences do you see between the entries in the 'parameter request list' option in this Request message and the same list option in the earlier Discover message?

The Parameter Request List in the Request message is identical to the one in the Discover message. Both messages request the same network configuration parameters, maintaining consistency throughout the DHCP exchange process.

Question 15. What is the source IP address in the IP datagram containing this ACK message? Is there anything special about this address? Explain.

As shown in Figure 23, the source IP address is 192.168.1.1, which is the DHCP server's IP address. This is the same source address used in the Offer message, confirming that the same DHCP server is completing the transaction.

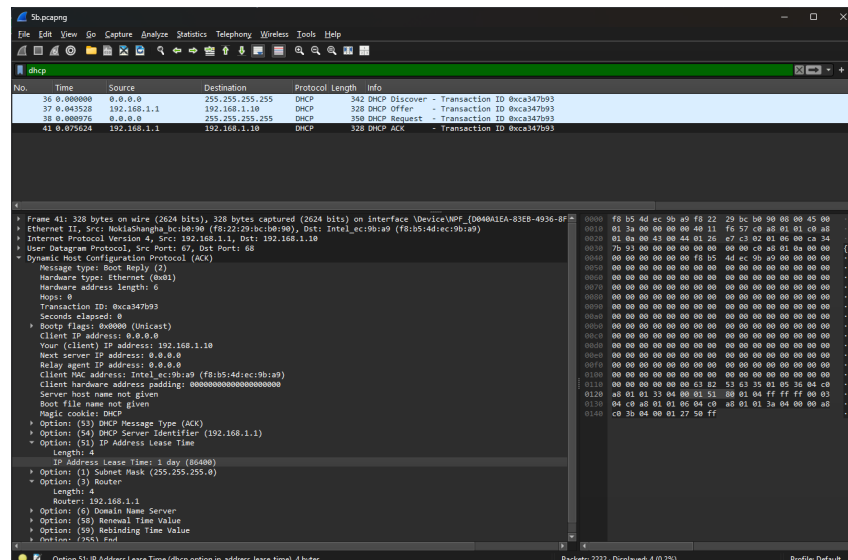


Figure 23: DHCP ACK message finalizing the IP address assignment. The server confirms all configuration parameters and specifies the lease duration for the assigned IP address.

Question 16. What is the destination IP address used in the datagram containing this ACK message. Is there anything special about this address? Explain.

From Figure 23, the destination IP address is 192.168.1.10. This is the IP address that has just been assigned to the client. The server sends the ACK to this address to confirm the assignment, and the client can now configure its network interface with this IP address.

Question 17. What is the name of the field in the DHCP ACK message (as indicated in the Wireshark window) that contains the assigned client IP address?

From Figure 23, the field name is "Your (client) IP address". This field in the DHCP header explicitly specifies the IP address being assigned to the client by the server.

Question 18. For how long a time (the so-called "lease time") has the DHCP server assigned this IP address to the client?

From Figure 23, the IP Address Lease Time is 1 day (86400 seconds). This lease time specifies how long the client may use the assigned IP address before it must either renew the lease or release the address back to the server.

Question 19. What is the IP address (returned by the DHCP server to the DHCP client in this DHCP ACK message) of the first-hop router on the default path from the client to the rest of the Internet?

From Figure 23, the router (default gateway) IP address is 192.168.1.1. This is the address the client will use as its default gateway for routing packets destined to networks beyond the local subnet.

2.3 Wireshark Lab: NAT

In this experiment, I analyzed network address translation (NAT) by examining packet traces captured on both sides of a NAT router. NAT allows multiple devices on a private network to share a single public IP address when communicating with the Internet. The trace files

nat-inside-wireshark-trace1-1.pcapng and nat-outside-wireshark-trace1-1.pcapng contain simultaneous captures from the LAN side and WAN side of the NAT router, respectively.

Question 1. What is the IP address of the client that sends the HTTP GET request in the nat-inside-wireshark-trace-1-1.pcapng trace? What is the source port number of the TCP segment in this datagram containing the HTTP GET request? What is the destination IP address of this HTTP GET request? What is the destination port number of the TCP segment in this datagram containing the HTTP GET request?

As shown in Figure 24, the client's source IP address is 192.168.10.11, which is a private IP address from the RFC 1918 address space. The TCP source port number is 53924. The destination IP address for this HTTP GET request is 138.76.29.8, which is the public IP address of the web server. The destination port number is 80, the standard HTTP port.

4	0.000000000	192.168.10.11	138.76.29.8	HTTP	396 GET / HTTP/1.1
6	0.003309856	138.76.29.8	192.168.10.11	HTTP	613 HTTP/1.1 200 OK (text/html)
8	0.200735320	192.168.10.11	138.76.29.8	HTTP	317 GET /favicon.ico HTTP/1.1
10	0.001667041	138.76.29.8	192.168.10.11	HTTP	555 HTTP/1.1 404 Not Found (text/html)

Frame 4: 396 bytes on wire (3168 bits), 396 bytes captured (3168 bits) on interface eth1, id 0 Ethernet II, Src: PCSSystemtec_89:c7:7c (08:00:27:89:c7:7c), Dst: PCSSystemtec_82:36:d7 (08:00:27:82:36:d7) Internet Protocol Version 4, Src: 192.168.10.11, Dst: 138.76.29.8 Transmission Control Protocol, Src Port: 53924, Dst Port: 80, Seq: 1, Ack: 1, Len: 330 Hypertext Transfer Protocol					
--	--	--	--	--	--

Figure 24: HTTP GET request as seen on the LAN side of the NAT router (inside trace). The source uses a private IP address 192.168.10.11 and ephemeral port 53924 to communicate with the web server.

Question 2. At what time is the corresponding HTTP 200 OK message from the webserver forwarded by the NAT router to the client on the router's LAN side?

As illustrated in Figure 25, the HTTP 200 OK message is forwarded by the NAT router to the client at time 0.003309856 seconds (relative to the start of the capture). This timestamp indicates when the response packet arrived at the client on the LAN side after being translated by the NAT router.

4	0.000000000	192.168.10.11	138.76.29.8	HTTP	396 GET / HTTP/1.1
6	0.003309856	138.76.29.8	192.168.10.11	HTTP	613 HTTP/1.1 200 OK (text/html)
8	0.200735320	192.168.10.11	138.76.29.8	HTTP	317 GET /favicon.ico HTTP/1.1
10	0.001667041	138.76.29.8	192.168.10.11	HTTP	555 HTTP/1.1 404 Not Found (text/html)

Frame 6: 613 bytes on wire (4904 bits), 613 bytes captured (4904 bits) on interface eth1, id 0 Ethernet II, Src: PCSSystemtec_82:36:d7 (08:00:27:82:36:d7), Dst: PCSSystemtec_89:c7:7c (08:00:27:89:c7:7c) Internet Protocol Version 4, Src: 138.76.29.8, Dst: 192.168.10.11 Transmission Control Protocol, Src Port: 80, Dst Port: 53924, Seq: 1, Ack: 331, Len: 547 Hypertext Transfer Protocol Line-based text data: text/html (12 lines)					
--	--	--	--	--	--

Figure 25: HTTP 200 OK response on the LAN side showing timestamp 0.003309856 s. The NAT router has translated the destination address back to the private IP 192.168.10.11.

Question 3. What are the source and destination IP addresses and TCP source and destination ports on the IP datagram carrying this HTTP 200 OK message?

From Figure 25, the addressing information for the HTTP 200 OK message on the LAN side is:

- Source IP Address: 138.76.29.8 (web server)
- Source Port: 80 (HTTP)
- Destination IP Address: 192.168.10.11 (client on LAN)
- Destination Port: 53924 (client's original source port)

Question 4. At what time does this HTTP GET message appear in the nat-outside-wireshark-trace1-1.pcapng trace file?

As shown in Figure 26, the HTTP GET message appears in the outside trace file at time 0.027356291 seconds. This represents when the NAT-translated packet is transmitted on the WAN side of the router.

4	0.027356291	10.0.1.254	138.76.29.8	HTTP	396 GET / HTTP/1.1
6	0.030625966	138.76.29.8	10.0.1.254	HTTP	613 HTTP/1.1 200 OK (text/html)
8	0.231400190	10.0.1.254	138.76.29.8	HTTP	317 GET /favicon.ico HTTP/1.1
10	0.233043313	138.76.29.8	10.0.1.254	HTTP	555 HTTP/1.1 404 Not Found (text/html)

Frame 4:	396 bytes on wire (3168 bits), 396 bytes captured (3168 bits) on interface eth0, id 0
Ethernet II, Src:	PCSSystemtec_43:65:cd (08:00:27:43:65:cd), Dst: PCSSystemtec_22:fd:74 (08:00:27:22:fd:74)
Internet Protocol Version 4, Src:	10.0.1.254, Dst: 138.76.29.8
Transmission Control Protocol, Src Port:	53924, Dst Port: 80, Seq: 1, Ack: 1, Len: 330
Hypertext Transfer Protocol	

Figure 26: HTTP GET request as seen on the WAN side of the NAT router (outside trace). The source IP address has been translated from the private address to the router's public IP 10.0.1.254.

Question 5. What are the source and destination IP addresses and TCP source and destination port numbers on the IP datagram carrying this HTTP GET?

From Figure 26, the addressing information on the WAN side is:

- Source IP Address: 10.0.1.254 (NAT router's public interface)
- Source Port: 53924 (preserved from original)
- Destination IP Address: 138.76.29.8 (web server)
- Destination Port: 80 (HTTP)

Question 6. Which of these four fields are different than in your answer to question 1 above?

Comparing Figure 24 and Figure 26, only the **source IP address** field has changed. The NAT router replaced the private source IP 192.168.10.11 with its own public IP 10.0.1.254. The source port, destination IP, and destination port remain unchanged.

Question 7. Are any fields in the HTTP GET message changed?

No fields in the HTTP GET message itself are changed. NAT operates at the network and transport layers, modifying IP addresses and maintaining port mappings, but the application-layer HTTP content remains identical. The HTTP headers and body are preserved byte-for-byte through the NAT translation.

Question 8. Which of the following fields in the IP datagram carrying the HTTP GET are changed from the datagram received on the local area network (inside) to the corresponding datagram forwarded on the Internet side (outside) of the NAT router: Version, Header Length, Flags, Checksum?

Of the fields listed, only the **Header Checksum** changes. Since the source IP address in the IP header is modified by NAT, the IP header checksum must be recalculated to maintain integrity. The Version, Header Length, and Flags fields remain unchanged.

Question 9. At what time does this message appear in the `nat-outside-wireshark-trace1-1.pcapng` trace file?

From Figure 26, the HTTP 200 OK response appears in the outside trace file at time 0.030625966 seconds. This is when the response packet from the web server reaches the WAN interface of the NAT router.

Question 10. What are the source and destination IP addresses and TCP source and destination port numbers on the IP datagram carrying this HTTP reply ("200 OK") message (as recorded in the `nat-outside-wireshark-trace1-1.pcapng` trace file)?

As shown in Figure 27, the addressing information for the HTTP 200 OK on the WAN side is:

- Source IP Address: 138.76.29.8 (web server)
- Source Port: 80 (HTTP)
- Destination IP Address: 10.0.1.254 (NAT router's public IP)
- Destination Port: 53924

4	0.027356291	10.0.1.254	138.76.29.8	HTTP	396	GET / HTTP/1.1
6	0.030625966	138.76.29.8	10.0.1.254	HTTP	613	HTTP/1.1 200 OK (text/html)
8	0.231400190	10.0.1.254	138.76.29.8	HTTP	317	GET /favicon.ico HTTP/1.1
10	0.233043313	138.76.29.8	10.0.1.254	HTTP	555	HTTP/1.1 404 Not Found (text/html)


```

Frame 6: 613 bytes on wire (4904 bits), 613 bytes captured (4904 bits) on interface eth0, id 0
  Ethernet II, Src: PCSSystemtec_22:fd:74 (08:00:27:22:fd:74), Dst: PCSSystemtec_43:65:cd (08:00:27:43:65:cd)
  Internet Protocol Version 4, Src: 138.76.29.8, Dst: 10.0.1.254
  Transmission Control Protocol, Src Port: 80, Dst Port: 53924, Seq: 1, Ack: 331, Len: 547
  Hypertext Transfer Protocol
  Line-based text data: text/html (12 lines)

```

Figure 27: HTTP 200 OK response on the WAN side destined to the NAT router's public IP 10.0.1.254. The router will translate this to the client's private address before forwarding to the LAN.

Question 11. What are the source and destination IP addresses and TCP source and destination port numbers on the IP datagram carrying the HTTP reply ("200 OK") that is forwarded from the router to the destination host in the right of Figure 1?

After NAT translation, the addressing information forwarded to the client on the LAN side is:

- Source IP Address: 138.76.29.8 (web server, unchanged)
- Source Port: 80 (HTTP, unchanged)
- Destination IP Address: 192.168.10.11 (client's private IP)
- Destination Port: 53924 (client's original port)

The NAT router maintains a translation table that maps the public IP and port combination to the corresponding private IP and port, enabling it to correctly translate the destination address of incoming responses back to the appropriate client on the LAN.

Code Availability

The complete implementation of this lab session, including all source code and related files, is available on GitHub. Please access the repository via <https://github.com/longlephantien/CN251-Lab>.